Przypomnienie poprzedniego wykładu

Czym jest agent i co potrafi?

Co już wiemy z poprzedniego wykładu?

Agent potrafi podejmować decyzje!

Agent ≠ Chatbot

Chatbot: Reaguje na polecenia.

Agent: Działa w celu i ma więcej możliwości.

Agent w środowisku

Agent postrzega swoje otoczenie, podejmuje decyzje i wykonuje działania.

Z jakich elementów się składa?

- Model (LLM)
- Instrukcje (prompt)
- Pamięć (kontekst)
- Narzędzia (tools)
- Cel i plan działania

Przykład: TupŻak

5 typów agentów

- Refleksyjny (prosty, jednokrokowy)
- Planistyczny (planowanie zadań)
- Wieloagentowy (współpraca)
- Z narzędziami (tool calling)
- Z pamięcią (długoterminowa)

Agent potrafi podejmować decyzje.

Ale skąd wie, jak powinien się zachować?

Dziś dowiemy się, jak to zrobić!

Pomysłów nam nie brakuje!

Wiemy, CO chcemy zbudować. Ale JAK?

OK,
wiem co chcę osiągnąć.
Jak to zrobić?
Jak to zrobić?
Prompt
(instrukcja)
LLM
(model)
Odpowiedź
(tekst / JSON)
Co muszę zrozumieć?
jak działa model?
jak pisać dobre prompty?
jakie ma ograniczenia?
kiedy potrzebne są narzędzia?
Kroki
1 Model i tokeny
Dzisiaj
2 Prompting i kontekst
Dzisiaj
3 Tool Calling
Następny wykład
Zaczynamy od modelu i promptów!

Co to jest model?

Dane wejściowe
- liczby
- tekst
- obrazy
- sygnały
- ...

Wejście (dane) → **MODEL** → **Wyjście** (predykcja / odpowiedź)

Wynik
- liczba
- klasa
- tekst
- obraz
- decyzja
- ...

Model to mechanizm (program z wyuczonymi parametrami), który na podstawie danych wejściowych generuje wynik.

Różne typy modeli — ten sam ogólny schemat

Model heurystyczny (reguły)

Ręcznie zapisane reguły

Dane wejściowe
np.
liczba pięter = 5
winda = tak

→ **Reguły (w kodzie)**
jeśli winda = tak
to skróć czas o 30%

→ **Wynik**
czas obsługi = 6 minut

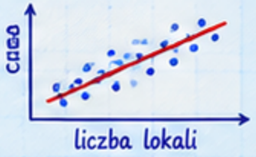
- Nie uczy się z danych.
- Działa dokładnie takie, jakie reguły zapiszemy.
- Dobre, gdy znamy jasne zasady.
- Ograniczone w bardziej złożonych sytuacjach.

Model uczenia maszynowego (ML)

Uczy się wzorców z danych

Dane wejściowe
liczba pięter = 5
winda = tak
pow. budynku = 1200
liczba lokali = 40
... .

→ **Model ML** (np. regresja, klasyfikacja)



→ **Wynik**
czas obsługi = 7.8 minut

- Uczy się zależności na podstawie przykładów.
- Dobrze radzi sobie z danymi liczbowymi.
- Wymaga danych treningowych.
- Generuje liczby, klasy, prawdopodobieństwa itp.

Duży model językowy (LLM)

Uczy się języka i generuje kolejne tokeny

Dane wejściowe
"Gdzie jest sala 204 w budynku C?"

→ **LLM (Transformer)**

Sala | 204 | znajduje | się | ...

kolejne generowane tokeny

→ **Wynik**
"Sala 204 znajduje się na drugim piętrze ... w budynku C."

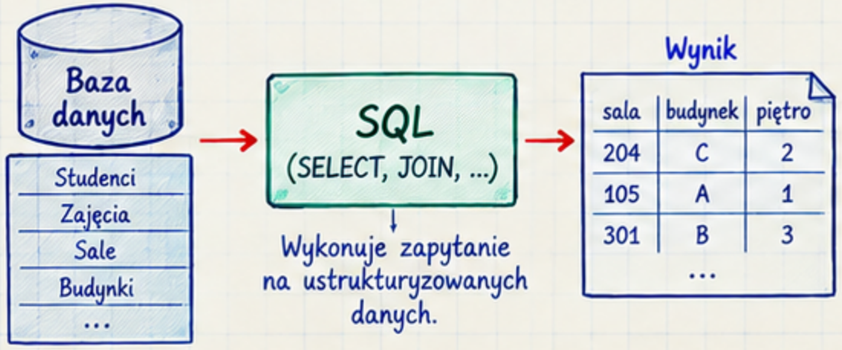
- Uczy się na ogromnych zbiorach tekstów.
- Rozumie kontekst i generuje kolejne tokeny.
- Może odpowiadać na pytania, streszczać, tłumaczyć, planować, wywoływać narzędzia itp.
- Działa na tekście (ale także obrazie, dźwięku itp. w wersjach multimodalnych).

Model to sposób przekształcania danych wejściowych w wynik.
Różne typy modeli uczą się z różnych danych i generują różne typy wyników.

SQL, ML, LLM — trzy różne sposoby pracy z danymi

SQL
Zapytanie do bazy danych

Zwraca dokładne dane, które już istnieją.



Przykład

```
SELECT sala, budynek, piętro
FROM sale
WHERE budynek = 'C' AND piętro = 2;
```

Cechy

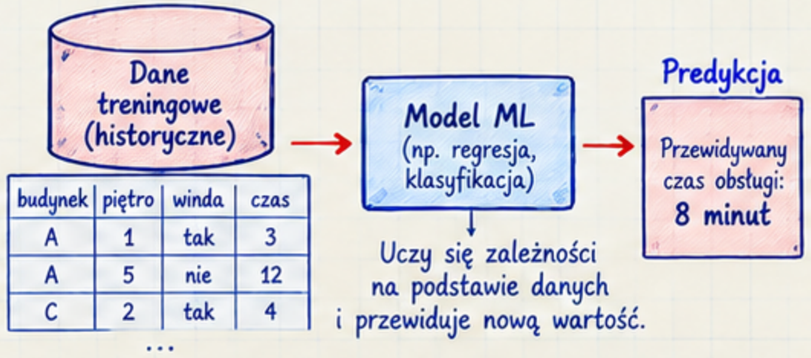
- ✔ dokładne, powtarzalne wyniki
- ✔ pracuje na ustrukturyzowanych danych
- ✔ szybkie i wydajne
- ✔ idealne do wyszukiwania, filtrowania, agregacji

Kiedy używać?

- gdy dane są w bazie
- gdy potrzebne są dokładne, aktualne informacje
- raporty, listy, statystyki
- logika biznesowa oparta na danych

ML
Predykcja na podstawie danych

Uczy się wzorców z danych i przewiduje wyniki.



Przykład

```
Wejście:  budynek = C, piętro = 6,
          winda = nie, liczba jednostek = 20

Wyjście:  przewidywany czas = 8.3 min
```

Cechy

- ✔ uczy się na podstawie danych
- ✔ dobry do predykcji i klasyfikacji
- ✔ wymaga danych treningowych
- ✔ działa na cechach (liczbowych, kategorycznych, ...)
- ✔ zwraca liczby, klasy lub prawdopodobieństwa

Kiedy używać?

- gdy chcemy przewidzieć wartość (np. czas dostawy)
- gdy mamy dane historyczne
- gdy masz dobrze zdefiniowane cechy
- problemy z liczbami, klasami, rankingiem

LLM
Rozumienie i generowanie języka

Pracuje na tekście i generuje kolejne tokeny.

Przykład

Prompt:
Jesteś asystentem Kampusu MiNi. Na podstawie danych o budynkach odpowiedz, gdzie jest sala 204 w budynku C. Jeśli nie masz informacji, powiedz, że nie wiesz.

Odpowiedź:
Sala 204 znajduje się na drugim piętrze w budynku C. ...

Cechy

- ✔ rozumie i generuje język naturalny
- ✔ radzi sobie z nieustrukturyzowanym tekstem
- ✔ może wyjaśniać, podsumowywać, klasyfikować, tłumaczyć
- ✔ wymaga dobrego promptu i kontekstu
- ✔ może generować strukturalne odpowiedzi (np. JSON)

Kiedy używać?

- gdy dane są w formie tekstu (pytania, dokumenty, maile)
- gdy potrzebna jest interpretacja i generowanie odpowiedzi
- gdy interakcja odbywa się w języku naturalnym
- gdy chcemy uzyskać wyjaśnienia, streszczenia, rekomendacje
- gdy potrzebna jest elastyczna logika i obsługa wielu przypadków

Te technologie się uzupełniają — w jednym systemie możemy używać SQL, ML i LLM razem.

Jak wybrać model?

Nie wybieramy najlepszego modelu, tylko model **wystarczający** dla konkretnego zadania i ograniczeń systemu.

ZADANIE

Co chcemy osiągnąć?

- odpowiadać na pytania
- analizować dane
- klasyfikować / wyciągać info
- generować tekst / podsumowania
- obsługiwać narzędzia (tools)

...

KRYTERIA WYBORU

Jakość

- trafność odpowiedzi
- rozumienie kontekstu
- zgodność z formatem

Koszt

- cena za 1M tokenów
- całkowity koszt
- dostępność darmowej wersji

Szybkość

- opóźnienie odpowiedzi
- przepustowość (liczba requestów)
- stabilność

Kontekst

- wielkość context window
- radzenie sobie z długimi dokumentami
- możliwość pracy z wieloma plikami

Możliwości

- structured output (JSON)
- tool calling / funkcje
- multimodalność (tekst, obraz, dźwięk)
- fine-tuning / własne dane

Prywatność

- lokalnie czy w chmurze
- dane wrażliwe
- zgodność z regulacjami
- kontrola nad danymi

PRZYKŁADOWE MODELE

Lokalne (np. przez Ollama)

Llama 3
Mistral

- działają na własnym sprzęcie
- brak kosztów za API
- większa kontrola nad danymi
- dobra jakość dla wielu zadań
- wymagają lokalnych zasobów

Kiedy wybrać?

- praca z danymi wrażliwymi
- brak dostępu do chmury
- prototypowanie / nauka
- mniejsze wymagania jakościowe

Modele chmurowe (ogólne)

GPT-4o
Claude 3.5 Sonnet

- bardzo wysoka jakość
- duże context window
- obsługa narzędzi i JSON
- multimodalność (tekst, obraz, dźwięk)
- wysoka dostępność i stabilność

Kiedy wybrać?

- gdy zależy nam na jakości
- złożone zadania i agenty
- praca z długimi dokumentami
- potrzebne narzędzia (tools)
- brak lokalnych zasobów

Modele lekkie / szybkie

GPT-4o mini
Llama 3 8B

- niższy koszt
- szybkie odpowiedzi
- dobra jakość dla prostszych zadań
- odpowiednie do klasyfikacji, ekstrakcji informacji

Kiedy wybrać?

- proste zadania
- duży wolumen zapytań
- aplikacje czasu rzeczywistego
- niska latencja
- kontrola kosztów

Modele wyspecjalizowane

Gemini (multimodalne)
Mistral (kod)
Embeddingi

- modele do konkretnych zastosowań
- multimodalne (obraz, dźwięk, wideo)
- modele kodu
- modele embeddingów (RAG)
- różne rozmiary i możliwości

Kiedy wybrać?

- zadania multimodalne
- generowanie lub analiza kodu
- tworzenie embeddingów do RAG
- specyficzne wymagania domenowe

Ten sam system może korzystać z kilku modeli — wybieramy odpowiedni model do konkretnej części zadania.

Context window — ile model może zapamiętać?

Context window to maksymalna liczba tokenów, które model może jednocześnie „widzieć" podczas jednego wywołania. Wszystko, co jest poza oknem, jest dla modelu niewidoczne.

CONTEXT WINDOW (np. 128k tokenów)

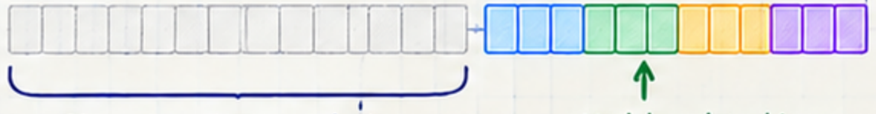
Różne modele — różne okna

Model	Context window
GPT-4o	128 000
Claude 3.5 Sonnet	200 000
Llama 3 (8B)	8 000
Llama 3 (70B)	128 000
Mistral Large	128 000
... ...	...

Co się dzieje, gdy przekroczymy limit?

Za dużo tokenów (np. 200 000) → ✗ Błąd
Model nie przyjmie całego kontekstu.

Model widzi tylko ostatnie tokeny



Starsze części mogą być pominięte lub ucięte ...!

Model widzi głównie tę część kontekstu.

Więcej kontekstu ≠ zawsze lepiej

✓ więcej informacji
✓ lepsze dopasowanie odpowiedzi
✓ możliwość podania przykładów

✗ większy koszt
✗ dłuższy czas odpowiedzi
✗ ryzyko nieistotnych informacji
✗ może pogorszyć jakość odpowiedzi

Przykład: pytanie o Kampus MiNi

Gdzie jest sala 204?

✗ Za dużo dokumentów (10 000 stron)

Cała dokumentacja kampusu (10 000 stron)
→ przekracza limit
→ błąd lub ucięcie istotnych informacji

✓ Wybrane fragmenty (tylko istotne)

Tylko informacje o budynkach i salach
→ mieści się w oknie
→ lepsza i szybsza odpowiedź

Jak zarządzać kontekstem?

1. Wysyłaj tylko istotne informacje (np. RAG).
2. Streszczaj dłuższe dokumenty.
3. Usuwaj niepotrzebną historię rozmowy.
4. Używaj wielu wywołań, jeśli potrzeba.
5. Pamiętaj o limitach wybranego modelu.

Prompt — jak rozmawiamy z modelem?

LOOP

Każda odpowiedź staje się częścią kontekstu do kolejnej tury.

Budujemy kontekst (prompt), model generuje odpowiedź, która może być kolejnym wejściem.

1. Budowa promptu

System prompt
Kim jesteś?
Jesteś asystentem Kampusu MiNi.
Styl, ograniczenia, format.

Kontekst / dane
Informacje potrzebne do zadania (np. opis budynku, dane z bazy, wynik narzędzia).

Instrukcja (user prompt)
Co masz zrobić?
Jakie są oczekiwania?
Czego nie wolno (np. nie zgaduj)?

Przykłady (opcjonalnie)
Zero-shot: brak przykładów
One-shot: 1 przykład
Few-shot: kilka przykładów

Całość połączona w jeden prompt (tekst) →

2. Prompt (tekst)

System:
Jesteś asystentem Kampusu MiNi. Pomagasz studentom w poruszaniu się po kampusie, sprawdzaniu planu zajęć, dostępie do zasobów. Odpowiadaj zwięźle, na podstawie dostarczonych danych. Nie zgaduj. Zwracaj wynik w formacie JSON.

Dane o budynku:
- nazwa: Gmach Główny
- poziomy: 8
- winda: nieznane
- wejście: od podwórka

Zadanie:
Na podstawie danych określ brakujące informacje i zwróć wynik w formacie JSON.

Przykład:
Pytanie: "Gdzie jest sala 204?"
Odpowiedź: {"type": "nawigacja", "confidence": 0.9}

Pytanie:
"Czy w budynku jest winda?"

Tokenizacja (tekst → tokeny) →

3. Tokeny

Jesteś	18342
asystentem	92817
Kampusu	37421
MiNi	516
.	13
Dane	8123
o	421
budynku	28451
:	29
poziomy	77219
:	29
8	367
⋮	

→

4. LLM

Model językowy (Transformer)

→

5. Odpowiedź

Wygenerowane tokeny (dekodowanie)

```
{
  "hasElevator": null,
  "confidence": 0.72,
  "reason": "Brak informacji w danych."
}
```

Parametry generacji

- temperatura
- top_p
- max_tokens
- itp.

Zero-shot — tylko instrukcja

Przypisz pytanie studenta do jednej z kategorii: NAWIGACJA, PLAN_ZAJĘĆ, ZASOBY.

Pytanie:
"Gdzie jest sala 204?"

→ "NAWIGACJA"

One-shot — jeden przykład

Przykład:
"Gdzie jest automat z kawą?" → "ZASOBY"

Teraz przypisz kategorię:
"Gdzie jest sala 204?"

→ "NAWIGACJA"

Few-shot — kilka przykładów

Przykłady:
"Gdzie jest automat z kawą?" → "ZASOBY"
"O której mam matematykę?" → "PLAN_ZAJĘĆ"
"Jak dojść do sali 105?" → "NAWIGACJA"

Teraz przypisz kategorię:
"Gdzie jest sala 204?"

→ "NAWIGACJA"

Structured Output — trzy formaty odpowiedzi

Pytanie: Podaj trzy miejsca warte odwiedzenia na kampusie Politechniki Warszawskiej.

1. Tekst w punktach

Oto trzy miejsca warte odwiedzenia na kampusie PW:

1. Gmach Główny – historyczny budynek z piękną architekturą.
2. Biblioteka Główna – spokojne miejsce do nauki.
3. Ogród na dachu BUW – świetny widok na panoramę Warszawy.

Zastosowanie:

- czytelna odpowiedź dla człowieka
- szybkie podsumowanie
- np. w interfejsie czatu

2. Markdown (ASCII)

```
# Trzy miejsca na kampusie PW

| Miejsce           | Opis                          |
|-------------------|-------------------------------|
| Gmach Główny      | Historyczny budynek           |
|                   | z piękną architekturą.        |
| Biblioteka Główna | Spokojne miejsce              |
|                   | do nauki.                     |
| Ogród na dachu BUW | Świetny widok na             |
|                   | panoramę Warszawy.            |
```

Zastosowanie:

- raporty i dokumentacja
- tabele, nagłówki, listy
- łatwe formatowanie i eksport

3. JSON (strukturalne dane)

```
{
  "places": [
    {
      "name": "Gmach Główny",
      "description": "Historyczny budynek
                      z piękną architekturą."
    },
    {
      "name": "Biblioteka Główna",
      "description": "Spokojne miejsce
                      do nauki."
    },
    {
      "name": "Ogród na dachu BUW",
      "description": "Świetny widok na
                      panoramę Warszawy."
    }
] }
```

Zastosowanie:

- integracja z aplikacją
- przetwarzanie danych
- np. zapis do bazy, dalsze narzędzia

Ale skąd agent ma to wiedzieć?

LLM sam z siebie nie ma dostępu do danych z uczelni.

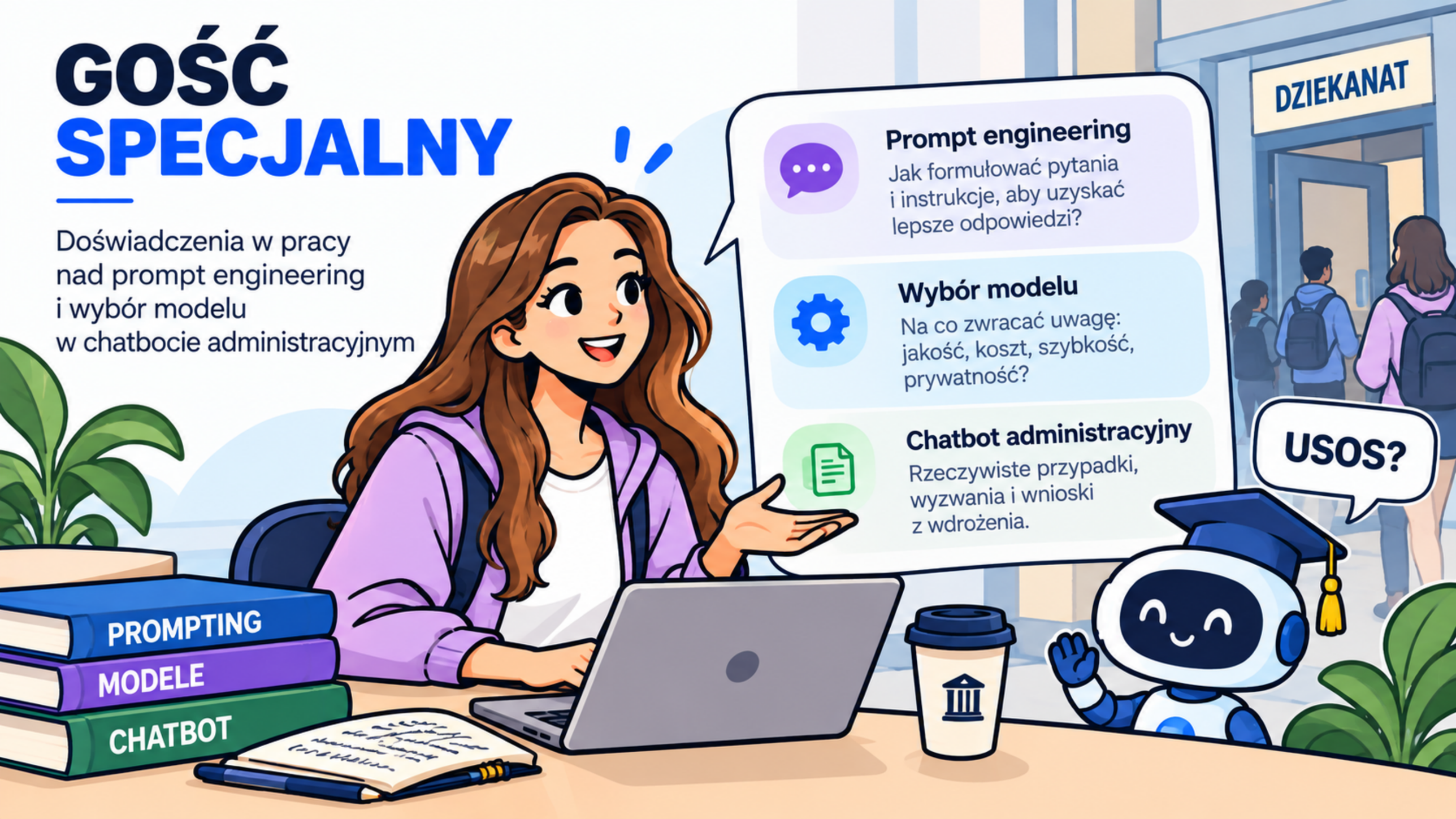
Następny wykład:

Tool Calling — jak agent zaczyna działać?

Dowiemy się, jak dać agentowi dostęp do narzędzi!

GOŚĆ SPECJALNY

Doświadczenia w pracy
nad prompt engineering
i wybór modelu
w chatbocie administracyjnym



Wykład 2

Modele i prompting w .NET

Jak agent rozumie nasze oczekiwania?

Marcin Sulecki
Wydział Matematyki i Nauk Informacyjnych
Politechnika Warszawska

Modele
jak działają?

Prompting
jak rozmawiać z modelem?

Kontekst
co model pamięta?

Strukturalne odpowiedzi
tekst, JSON, Markdown

Wybór modelu
jakość, koszt, możliwości